

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

*I _____K. Deepthi (192525343) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: K. Deepthi

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Set 5

1. A compiler design team is developing a semantic analyzer for a newly proposed programming language used in banking applications. The compiler must process arithmetic expressions, conditional statements, and nested loops while maintaining datatype consistency. The team decides to use Syntax-Directed Definitions to attach semantic rules with grammar productions and construct syntax trees for efficient translation. During testing, inherited attributes are not correctly propagated in nested expressions, leading to incorrect semantic outputs. Analyze how L-attributed definitions and top-down translation techniques can resolve these semantic dependency issues and improve compiler accuracy.

Answer:

Case Study – L-Attributed Definition

A banking application compiler needs to check datatype consistency in arithmetic expressions, conditions, and nested loops. Initially, incorrect propagation of inherited attributes caused semantic errors.

Problem

In nested expressions, information such as datatype and scope was not passed correctly from parent nodes to child nodes.

Solution

Using L-Attributed Definitions, inherited attributes are passed from the parent or left sibling to the child. This supports left-to-right, top-down translation.

Example:

$E \rightarrow T E'$

$E'.inh = T.type$

The compiler calculates the required attribute before processing the next child.

Top-Down Translation

In top-down translation, the compiler starts from the root of the syntax tree and moves towards the leaf nodes. Inherited attributes are calculated before processing the child nodes.

Handling Nested Expressions

For an expression such as:

$total = (principal + interest) * tax;$

the compiler passes the required datatype information to each nested expression and checks whether the operations are valid.

Handling Conditional Statements

For conditions such as:

$if (balance > 50000)$

the compiler checks whether `balance` and `50000` have compatible types and ensures that the result of the condition is Boolean.

Handling Nested Loops

In nested loops, inherited attributes can carry **scope and datatype information** from the outer block to the inner block. This helps the compiler correctly identify variables and their types.

Benefits

- Correct inherited attribute propagation
- Accurate datatype checking
- Proper scope management

- Correct handling of nested expressions
- Efficient syntax-tree traversal
- Fewer semantic errors
- Improved compiler reliability

Conclusion

Thus, combining L-Attributed Definitions with top-down translation solves the inherited attribute propagation problem. It provides correct semantic analysis, datatype consistency, and reliable translation, which are important for a banking application compiler.

2. A software engineering company is implementing a compiler for a robotics control language where fast semantic evaluation is essential for real-time execution. The parser currently generates complete parse trees, resulting in excessive memory usage and delayed intermediate code generation. To optimize performance, the compiler architect recommends constructing syntax trees and applying bottom-up evaluation of S-attributed definitions. However, some semantic rules are still producing inconsistent results during reductions. Examine how syntax tree construction and bottom-up attribute evaluation improve efficiency and explain why synthesized attributes are better suited for LR parsing environments.

ANSWER:

Syntax Tree Construction and Bottom-Up Evaluation

Scenario:

A software engineering company is developing a compiler for a robotics control

language. Since robotic systems require fast execution, the compiler must perform semantic analysis and generate intermediate code efficiently.

Problems in the Existing System

- The parser generates complete parse trees, which require more memory.
- Large parse trees increase processing time.
- Intermediate code generation is delayed until the complete tree is processed.
- Some semantic rules give inconsistent results during LR reductions.

Proposed Solution

The compiler architect recommends using syntax trees with S-attributed definitions and bottom-up evaluation.

1. Syntax Tree Construction

- A syntax tree stores only the important operators and operands.
- Unnecessary grammar symbols are removed.
- This reduces memory usage and makes later processing faster.

2. Bottom-Up Evaluation

- Attributes are evaluated when a production is reduced.
- The values of child nodes are calculated first and then used to calculate the parent node.
- This matches the natural bottom-up operation of LR parsers.

3. S-Attributed Definitions

- S-attributed definitions use only synthesized attributes.
- A synthesized attribute gets its value from the attributes of its children.
- Example:

$$E \rightarrow E1 + T \{ E.val = E1.val + T.val \}$$

4. Why Suitable for LR Parsing

- LR parsers work from leaves toward the root through shift-reduce operations.
- During a reduction, all required child attributes are already available.
- Therefore, synthesized attributes can be evaluated immediately without passing information from parent to child.

5. Efficiency Improvement

- Less memory is required because a complete parse tree is not maintained.
- Semantic evaluation can happen during parsing.
- Intermediate code can be generated earlier.
- This improves the compiler's speed and responsiveness, which is important for real-time robotics.

Conclusion

Using syntax trees and S-attributed definitions provides an efficient solution for the robotics compiler. Syntax trees reduce memory overhead, while bottom-up synthesized attribute evaluation fits naturally with LR parsing. Therefore, it reduces semantic inconsistencies and enables faster intermediate-code generation and real-time compiler performance.

3. A university research laboratory is designing a compiler capable of handling user-defined datatypes and multidimensional arrays. The semantic analyzer must determine whether two complex datatype declarations are equivalent before assignment operations are permitted. The current implementation relies only on datatype names, causing structurally similar datatypes to be rejected incorrectly. The research team plans to enhance the compiler using structural type equivalence and explicit type conversion rules. Investigate how equivalence of type

expressions and type systems contribute to reliable semantic analysis in such applications.

ANSWER:

Type Equivalence and Type Systems in Semantic Analysis

Scenario

A university research laboratory is developing a compiler that supports user-defined datatypes and multidimensional arrays. Before allowing assignment operations, the semantic analyzer must check whether the source and destination datatypes are compatible.

Problem

The existing compiler checks only datatype names. Therefore, two datatypes with the same internal structure but different names may be incorrectly rejected.

Example:

type A = array [10] of integer;

type B = array [10] of integer;

Although A and B have the same structure, name-based checking may consider them different.

Proposed Solution

1. Structural Type Equivalence

- Two types are considered equivalent if their structures are identical, even if their names are different.
- For example, two arrays with the same dimensions and element type can be treated as equivalent.
- This is useful for user-defined types and complex nested structures.

2. Equivalence of Type Expressions

- A type expression represents the construction of a datatype.

- For arrays, the type expression includes size, dimensions, and element type.

Example:

Array [10] of integer

can be compared with another type expression having the same structure.

3. Type Systems

- A type system defines rules for determining which operations are valid.
- It prevents invalid assignments such as assigning a float value to an incompatible structured type.
- It also provides rules for type conversion when conversion is permitted.

4. Explicit Type Conversion

- Some compatible types can be converted explicitly.

Example:

integer $\rightarrow$ float

- The compiler should perform the conversion only when the type system allows it, avoiding unsafe assignments.

5. Benefits for Multidimensional Arrays

- The analyzer can compare each array dimension and its element type.
- For example:

Array [5][10] of integer

is structurally different from:

Array [5][10] of float

- This prevents incorrect assignments and improves type safety.

6. Early Error Detection

- The semantic analyzer checks type equivalence before code generation.

- This helps detect incompatible assignments at an early stage.
- It reduces runtime errors and improves the reliability and safety of the compiler.

Conclusion

- Structural type equivalence allows the compiler to recognize datatypes based on their actual structure rather than only their names. A well-defined type system and explicit conversion rules ensure that assignments and operations are performed safely. Together, they make semantic analysis more accurate, flexible, and reliable for user-defined datatypes and multidimensional arrays.
4. A mobile application development company is creating a source-to-source translator that converts high-level programs into optimized intermediate representations. The translator uses syntax-directed translation schemes to evaluate semantic actions during parsing. During testing, expressions containing mixed datatypes such as integer, float, and boolean values generate semantic inconsistencies across different modules. The development team decides to introduce a simple type checker with conversion handling mechanisms. Analyze how type checking and type conversion policies improve translation correctness and prevent semantic errors during intermediate code generation.

ANSWER:

Type Checking and Type Conversion

Scenario

A mobile application development company is developing a source-to-source translator that converts high-level programs into optimized intermediate representations. It uses Syntax-Directed Translation (SDT) to perform semantic actions during parsing.

Problem

Expressions containing integer, float, and boolean datatypes produce inconsistent results. For example, operations such as `int + float` or `int + boolean` may cause semantic errors during intermediate code generation.

Proposed Solution

1. Type Checking

- The compiler checks whether operands have compatible datatypes before performing an operation.
- Example:
- `int + float` → valid

`int + boolean` → type error

- This prevents invalid operations from reaching code generation.

2. Type Conversion

- When compatible datatypes are used, the compiler can automatically convert one type into another.
- Example:

`int + float`

The integer can be converted to float before performing the addition.

3. Conversion Policies

- Implicit conversion: Compiler automatically converts the datatype.
- Explicit conversion: Programmer specifies the required conversion.
- These policies ensure conversions happen only when they are safe and meaningful.

4. Semantic Actions

- Type information can be attached to grammar symbols.
- During parsing, semantic actions check the operand types and insert required conversion operations.
- This makes intermediate code generation more consistent.

5. Prevents Semantic Errors

- Invalid operations are detected during compilation instead of runtime.
- For example, a boolean value cannot be directly added to an integer.
- This improves the correctness and reliability of the translated program.

6. Improves Intermediate Code Generation

- Correct type information helps the translator generate appropriate intermediate instructions.
- Unnecessary conversions can also be avoided, improving the efficiency of the generated code.

Conclusion

Type checking ensures that operations are performed only on compatible datatypes, while type conversion handles valid mixed-type expressions. Together, they make syntax-directed translation more consistent, accurate, and reliable, while preventing semantic errors during intermediate code generation.

5. A compiler engineer is developing a parser for an artificial intelligence programming language that heavily relies on mathematical expressions and recursive function calls. The parser uses bottom-up parsing techniques to evaluate semantic rules and generate syntax trees. However, incorrect ordering of attribute evaluation causes dependency conflicts between inherited and synthesized attributes. To address this problem, the

engineer considers redesigning the semantic rules using L-attributed definitions. Evaluate how top-down translation, bottom-up evaluation, and proper attribute dependency management contribute to efficient syntax-directed translation and semantic correctness.

ANSWER:

L-Attributed Definitions in Syntax-Directed Translation Problem

A compiler for an AI programming language handles mathematical expressions and recursive function calls. The parser uses bottom-up parsing to construct syntax trees and evaluate semantic rules. However, incorrect ordering of inherited and synthesized attributes creates dependency conflicts, leading to incorrect semantic results.

Use of L-Attributed Definitions

The compiler can redesign its semantic rules using L-attributed definitions. In an L-attributed definition, an inherited attribute of a symbol can depend only on:

- Attributes of its parent.
- Attributes of symbols to its left.
- Inherited attributes of the parent.

This ensures that attributes can be evaluated in a valid left-to-right order.

Top-Down Translation

Top-down translation processes the input from the start symbol toward the leaves of the syntax tree. It is useful for:

- Passing inherited attributes downward.
- Providing contextual information to child nodes.
- Managing function parameters and variable types.
- Reducing dependency conflicts.

For example, the type or scope information of a function can be passed from the parent node to its parameters and statements.

Bottom-Up Evaluation

Bottom-up evaluation processes child nodes before their parent nodes. It is especially suitable for synthesized attributes.

For example:

$$E \rightarrow E1 + T$$

$$E.val = E1.val + T.val$$

Here, $E1.val$ and $T.val$ are calculated first, and then $E.val$ is obtained. This is naturally compatible with bottom-up parsing.

Attribute Dependency Management

Proper dependency management ensures that an attribute is evaluated only after all attributes on which it depends are available. A dependency graph can be used to identify the correct evaluation order.

For example:

Inherited attributes $\rightarrow$ Child processing $\rightarrow$ Synthesized attributes

This prevents circular dependencies and incorrect semantic values.

Effect on the AI Compiler

Using L-attributed definitions provides:

- Efficient left-to-right attribute evaluation.
- Correct propagation of contextual information.
- Better handling of recursive function calls.
- Fewer dependency conflicts.
- Correct construction of syntax trees.
- Improved semantic checking during translation.

Improved Error Detection

L-attributed definitions also help the compiler detect semantic errors at an earlier stage. For example, incorrect data types, undeclared variables, or invalid function parameters can be identified while attributes are being propagated. This prevents wrong semantic information from reaching the intermediate code generation phase and improves the overall reliability of the compiler.

Conclusion

The case study shows that top-down translation is effective for inherited attributes, while bottom-up evaluation is effective for synthesized attributes. By using L-attributed definitions and a proper dependency order, the compiler can perform syntax-directed translation efficiently and maintain semantic correctness even for complex mathematical expressions and recursive functions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

